Monoids: an introduction to lawful structures

George Wilson

george@wils.online

13th October 2026

Outline

Let's look at:

- Basic Haskell syntax
- A simple example type class
- The Monoid type class (including laws)
- Examples of monoid instances
- Why laws are helpful

Values and types

```
x :: Int
x = 5 + 10
```

Strings and concatenation

```haskell
name :: String
name = "BFPG"

greeting :: String
greeting = "Hello, " ++ name ++ "!"
```

Strings and concatenation

```haskell
name :: String
name = "BFPG"

greeting :: String
greeting = "Hello, " ++ name ++ "!"
```

```
> greeting
-- "Hello, BFPG!"
```

Functions on strings

```
greet :: String -> String
greet n = "Hello, " ++ n ++ "!"
```

Functions on strings

```haskell
greet :: String -> String
greet n = "Hello, " ++ n ++ "!"
```

```haskell
> greet name
-- "Hello, BFPG!"
```

Custom data types

```
data TrafficLight
  = Red
  | Yellow
  | Green
```

Our first type class

```
class Show a where
  show :: a -> String
```

Our first type class

```haskell
class Show a where
  show :: a -> String

instance Show TrafficLight where
  show tl =
    case tl of
      Red -> "Red"
      Yellow -> "Yellow"
      Green -> "Green"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```
> greet Yellow
-- "Hello, Yellow!"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```
> greet Yellow
-- "Hello, Yellow!"

> greet 5
-- "Hello, 5!"
```

Functions with type class constraints

```haskell
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```haskell
> greet Yellow
-- "Hello, Yellow!"

> greet 5
-- "Hello, 5!"

> greet (++)
-- Error: No Show instance for
--          (String -> String -> String)
```

Monoid

```
class Monoid m where
  mappend :: m -> m -> m
  mempty  :: m
```

Monoid

```
class Monoid m where
  mappend :: m -> m -> m
  mempty  :: m
instance Monoid String where
  mappend s t = s ++ t
  mempty = ""
```

Monoid laws

Left identity:

```
-- for all x.
mappend mempty x == x
```

Right identity:

```
-- for all x.
mappend x mempty == x
```

Associativity:

```
-- for all x y z.
mappend x (mappend y z) == mappend (mappend x y) z
```

Thanks for listening!